

Simple Talent Tree Ui

Easy way to create simple talent trees

Thank you for purchasing Simple Talent Tree Ui Asset to enhance your Unity projects.

We appreciate your trust in our asset, and we're excited to help you create engaging talent tree systems for your games.

Introduction

Welcome to Simple Talent Tree Ui Asset documentation. This asset provides a user-friendly and customizable Talent Tree Interface system for your Unity projects. Easily create and manage talent trees for character progression, skill unlocks, and more.

We understand that questions may arise as you use our asset, and we're here to assist you every step of the way. If you encounter any issues, seek clarification, or simply wish to share feedback, please don't hesitate to reach out to us via email at lonelyitguystudio@gmail.com. We are committed to providing prompt and helpful support to ensure your success with our asset.

This documentation will guide you through the installation, usage, and customization of the Simple Talent Tree Ui.

Content

- Introduction
- Features
- Installation
- Usability
- Classes

Features

- Custom editor to design a Talent Tree
- Regular and Random tree generation
- Ui Tooltip
- Ui Panel Zoom
- Runtime Tree Manager System
- Runtime Save System
- 10 Images to use as icon
- 1 Custom material
- Compatible with Screen Space Overlay and Camera

Installation

Unity Version Compatibility

Simple Talent Tree Ui was initially developed for 2022.3.37f1, but we have ensured its compatibility with all subsequent Unity versions.

Please note that we do not offer official support for Unity versions released before the 2022.3.37f1 LTS lifecycle. Installing the asset in such a version is at your own risk.

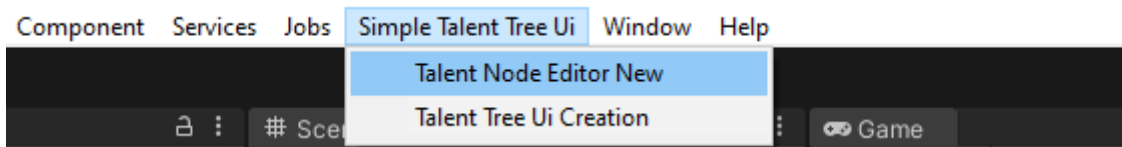
Installation Process

After downloading and importing the package to your project, you will find a new directory named 'SimpleTalentTreeUi' in your assets. You are free to relocate this entire folder to a different location within your assets directory. However, it is crucial to maintain the folder's structure, keeping all subdirectories unchanged. The one exception is that you have the option to remove the entire 'SampleScenes' directory if you do not require the provided examples.

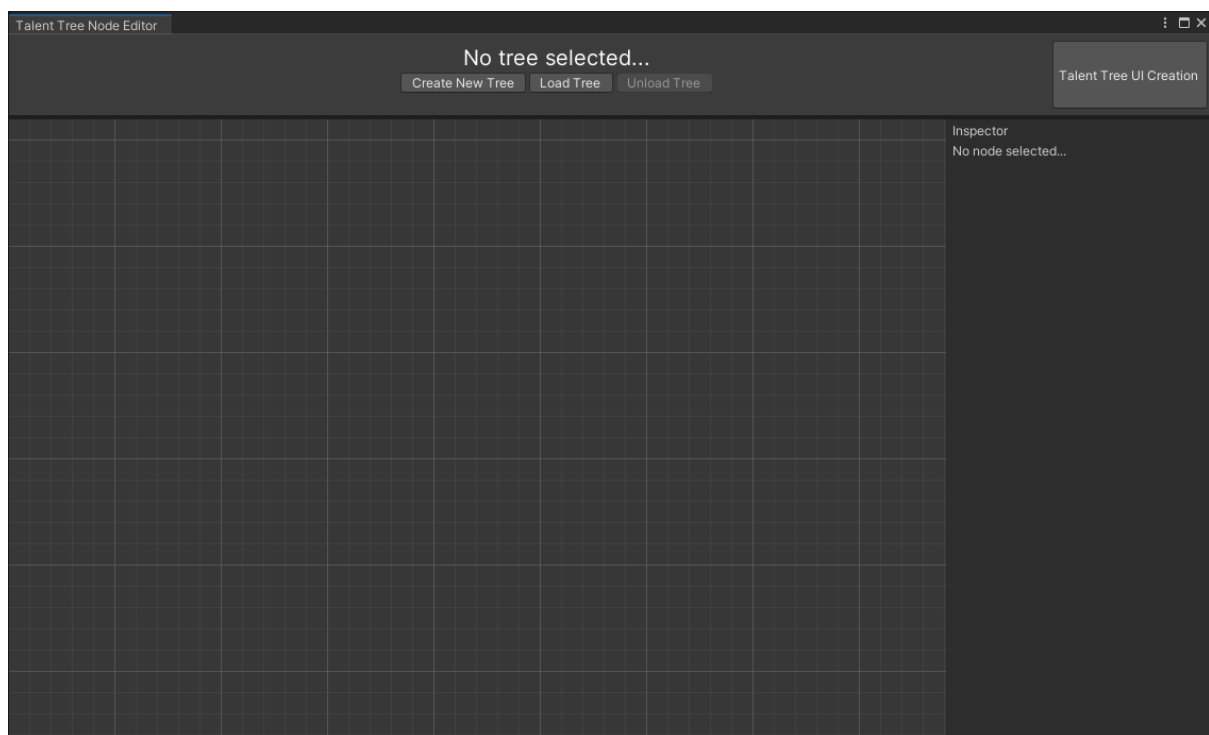
Usability

Talent Tree Creation

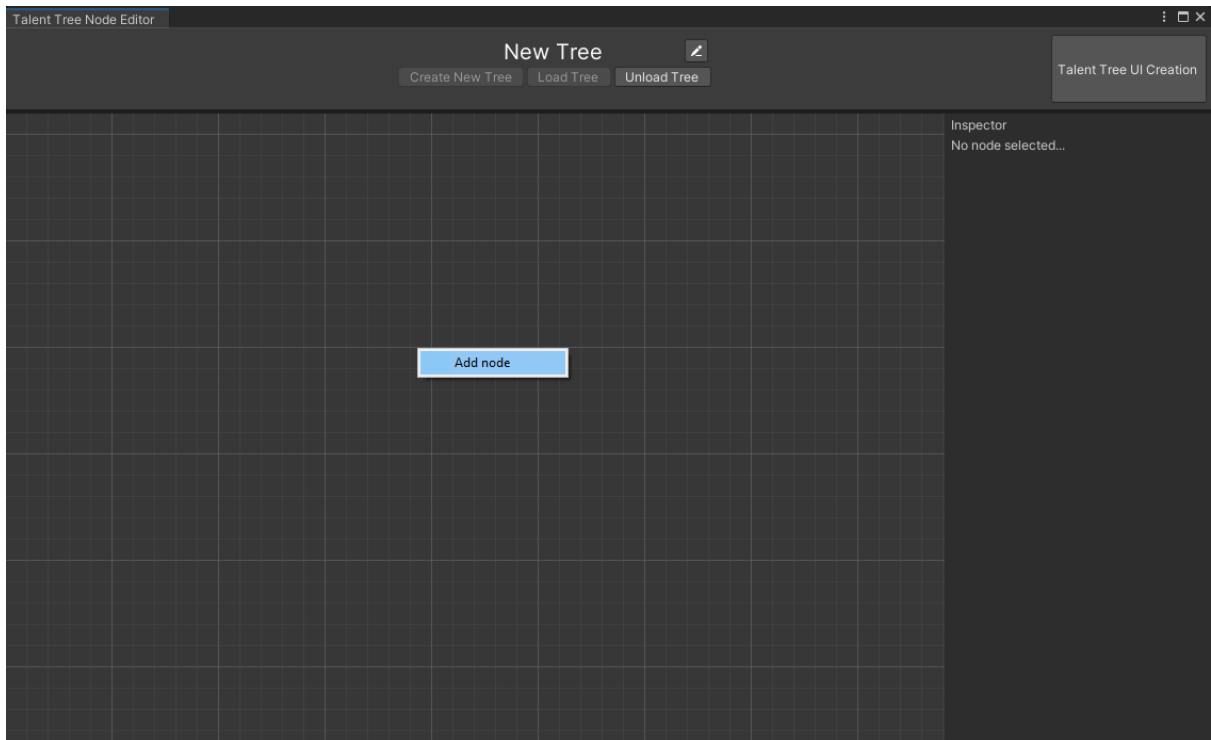
Initiate the creation of your talent tree by navigating to the SimpleTalentTreeUi menu in the Unity top bar. Here, you can craft your talent tree design using the node editor.



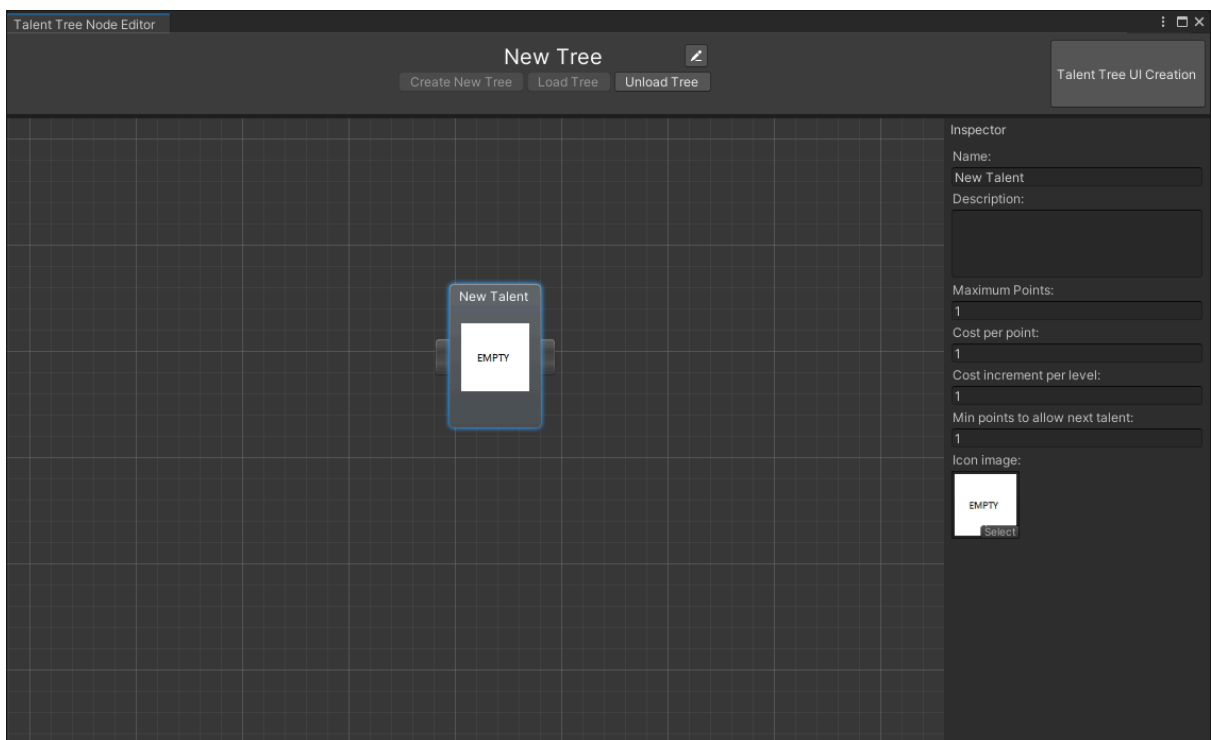
Upon accessing the menu, you will be presented with this window for creating and editing talent tree nodes.



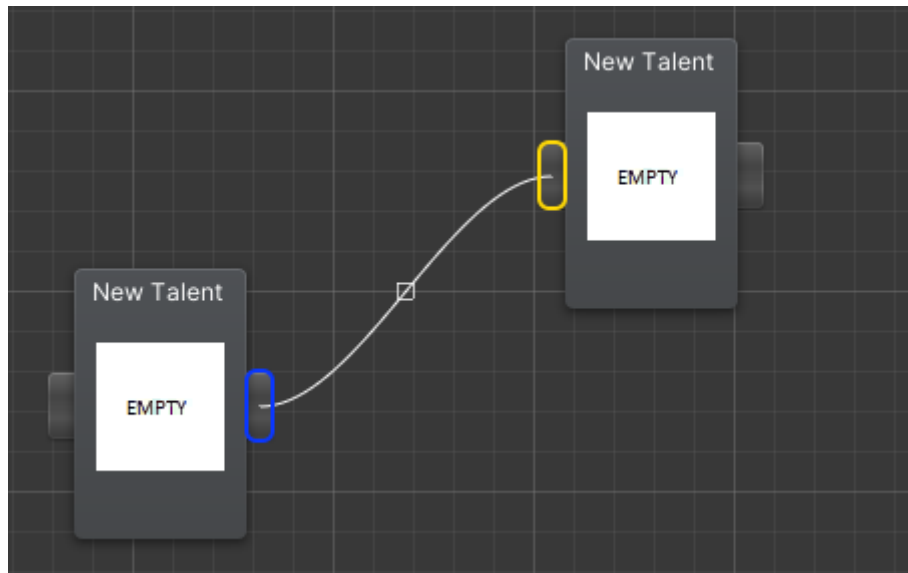
After clicking 'New Tree,' a new tree structure will be generated with the default name 'New Tree.' You can modify the name by clicking the 'Edit' button next to the tree name.



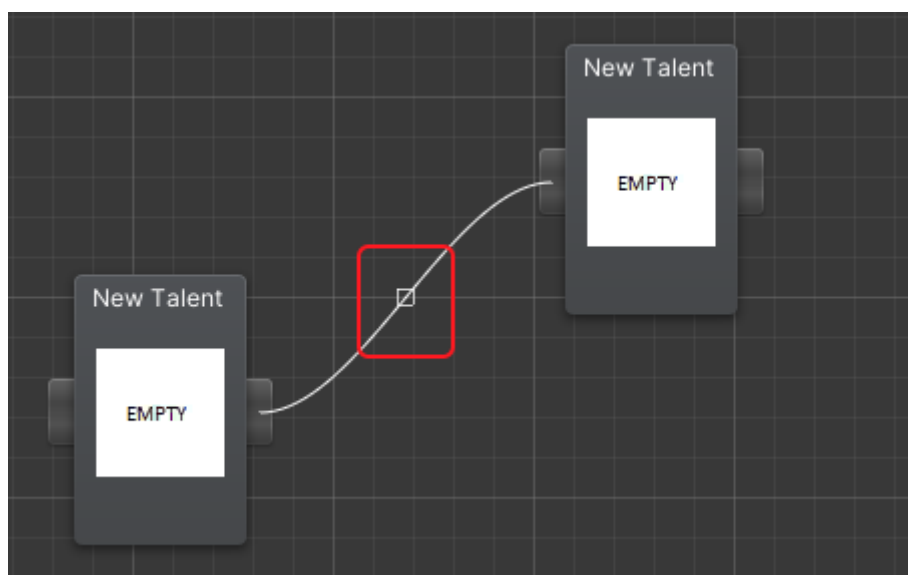
Right Click to create a new Node. When a new node is generated, its values are presented in the inspector located on the right side of the window. Here, you can modify the attributes related to the talent you wish to create.



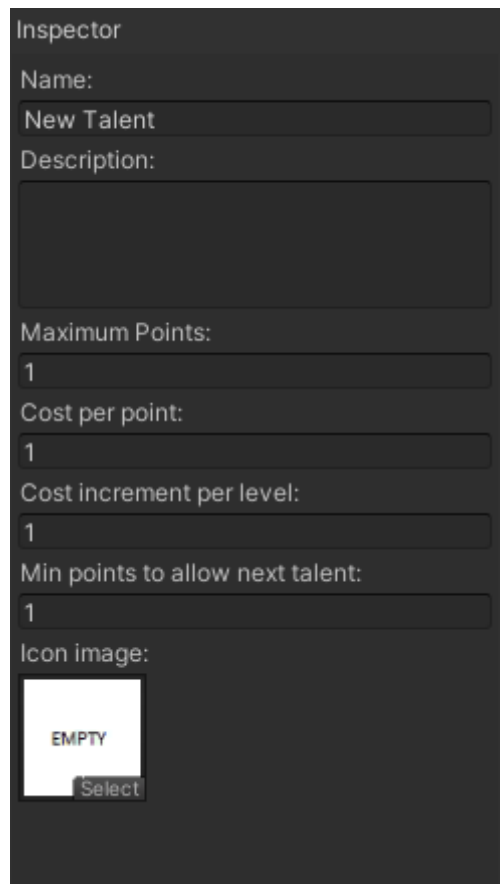
Once you have multiple talent nodes, you can establish connections between them. To create a connection, you must first click on the [blue](#) exit point depicted in the image and link it to the [yellow](#) entry point.



Connections between talents can be removed by clicking on the square located in the middle of the line



Node Inspector

A dark-themed user interface for a 'Node Inspector'. It contains several input fields and a button. The fields are labeled 'Name:', 'Description:', 'Maximum Points:', 'Cost per point:', 'Cost increment per level:', and 'Min points to allow next talent:'. Each of these fields contains the number '1'. The 'Icon image:' field shows a small square placeholder with the word 'EMPTY' and a 'Select' button below it.

Inspector

Name:
New Talent

Description:

Maximum Points:
1

Cost per point:
1

Cost increment per level:
1

Min points to allow next talent:
1

Icon image:

Select

[Name:](#) Attribute used to display the talent name in the user interface

[Description:](#) Attribute used to display the talent description in the user interface

[Maximum Points:](#) Attribute used to control the total amount of points for that talent.

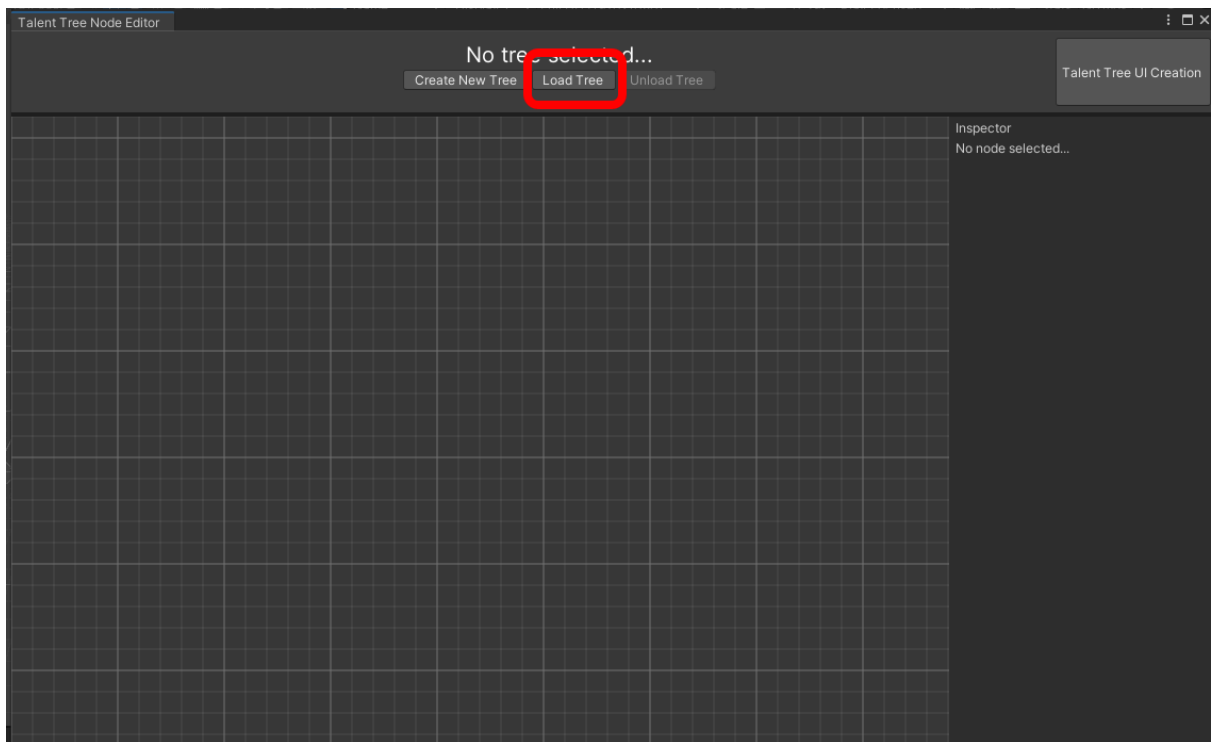
[Cost per point:](#) Attribute used to control the amount of points spent for each talent level.

[Cost increment per level:](#) Attribute used to control the increment per level of the amount of points spent for each talent level.

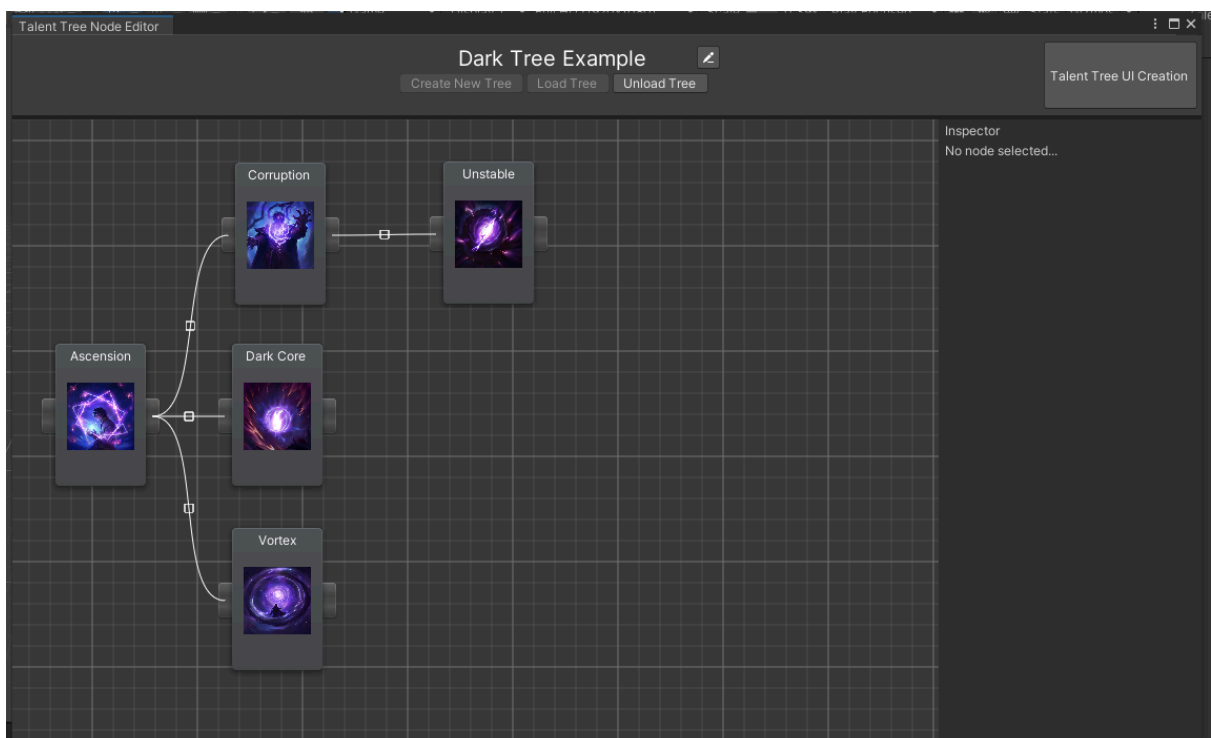
[Min points to allow next talent:](#) Attribute used to control the minimum amount of points needed of the current node to unlock the next nodes connected to it.

[Icon Image:](#) Attribute used to control the image displayed in the user interface.

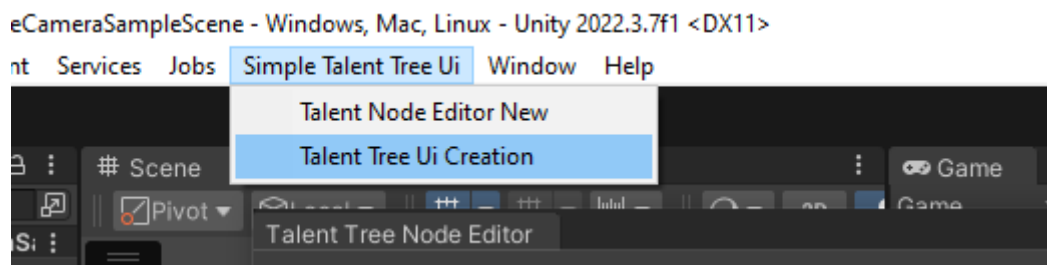
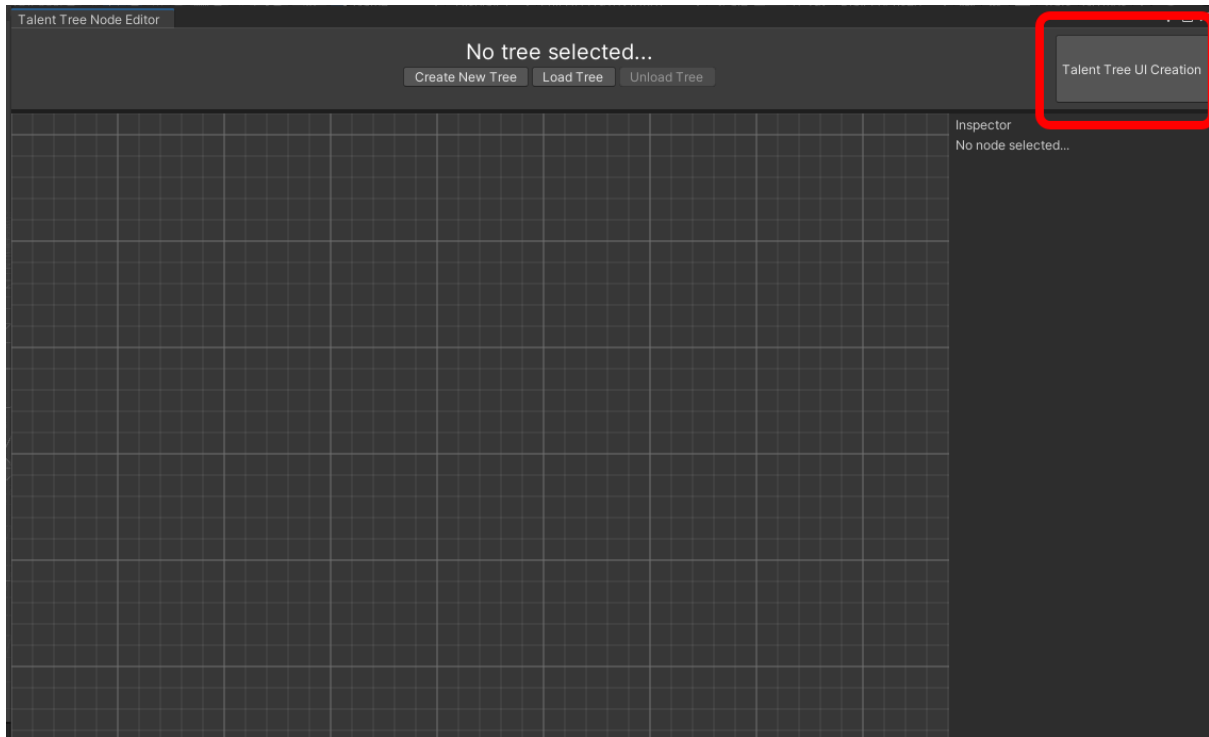
Loading a pre built tree



Click on the "Load Tree" button located in the center of the node editor window. After clicking the button, a file explorer window will open. There, you need to locate your previously created TalentTreeSO. The default folder path for creation is in SimpleTalentTree\ScriptableObjects\TalentTreeSOs.



Creating the Interface



Located in the button at the upper right corner of the node editing window or in the top menu of Unity, you will have access to the interface creation menu, “Talent Tree Ui Creation Window”

Talent Tree Ui Creation Window

Talent Tree Ui Creation

Open Tree Editor

References Settings

Talent Tree:

None (Talent Tree SO)

Ui Parent Transform:

None (Rect Transform)

Ui Node Prefab:

None (Game Object)

Ui Line Prefab:

None (Game Object)

Talent Manager (optional):

None (Game Object)

Add ToolTip to Ui:

☐

Line/Border disabled color:

Line/Border enabled color:

Regular Ui Generation

Clear UI Before Regular Generation:

☐

Generate Tree Ui

Random Ui Generation

Spawn Distance Radius:

80

Start Point Range:

X 400 Y 400

Random Seed:

0

Clear UI Before Random Generation

☐

Generated Seed:

0

Generate Random Tree Ui

Reference Settings

[Talent Tree:](#) TreeSO that will be used to generate the tree interface.

[Ui Parent Transform:](#) Hierarchy Transform Parent to spawn the talent nodes.

[Ui Node Prefab:](#) Node prefab that will be used to generate the tree interface.

[Ui Line Prefab:](#) Line render prefab that will be used to generate the tree interface.

[Talent Manager \(optional\):](#) Talent Manager class reference to be used in runtime to control talent leveling, point spending, etc.

[Add ToolTip to Ui:](#) Enable Node ToolTip on mouse over in runtime.

[Line/Border disabled color:](#) Disabled color for nodes and lines.

[Line/Border enabled color:](#) Enabled color for nodes and lines.

Regular Ui Generation

[Clear UI Before Regular Generation:](#) Toggle to clear or retain the currently generated nodes/lines inside the UI Parent Transform before initiating a new generation.

[Generate Tree Ui Button:](#) Button to generate the Talent Tree Interface following the design created inside the Talent Tree Node Editor Window.

Random Ui Generation

Important note: This type of generation doesn't adhere to the design created inside the Talent Tree Node Editor Window; instead, it will randomly position nodes following the parameters below. Additionally, it was designed for use with a small circular node prefab design. An example of the node is located inside the prefab folder.

[Spawn Distance Radius:](#) The radius distance between each node point.

[Start Point Range:](#) The range used to randomly position the first point.

[Random Seed:](#) The seed used to generate the tree based on a previously liked seed pattern.

[Clear UI Before Regular Generation:](#) Toggle to clear or retain the currently generated nodes/lines inside the UI Parent Transform before initiating a new generation.

[Generated Seed:](#) The seed created during the current random tree generation.

[Generate Random Tree Ui Button:](#) Button to generate a new random Talent Tree Interface following the parameters.

Classes

Talent Tree Manager class:

InitializeNewTreeFromEditor(string treeID)

- This method initializes a new talent tree based on the provided `treeID`. If the tree with the specified `treeID` already exists, it clears its talents and sets it as the active tree. If the tree does not exist, a new one is created and added to the list of talent trees. After initialization, the method triggers the `OnTalentActivated` event.

Save()

- This method saves the current state of the `TalentManager`, including all the talent trees and their respective talents. It returns a `TalentManagerSaveData` object, which contains the save data for the manager, including the talent trees and their associated talents.

Load(object data)

- This method loads the state of the `TalentManager` from the provided save data. It iterates through the list of talent trees in the saved data, finds the corresponding talent tree in the current instance, and loads its state. After loading all relevant data, it triggers the `LoadFinished()` method to indicate that loading is complete.

FindTreeByID(string treeId)

- This method searches for a talent tree by its ID (`treeId`). It returns the `TalentTree` object if found; otherwise, it returns `null`. This method is essential for locating specific talent trees within the manager, particularly during loading and initialization processes.

TalentTree Class:

Initialize(TalentTreeSO newTalentTreeSO)

- This method initializes the talent tree using a **TalentTreeSO** (Scriptable Object). It sets the **treeld** based on the data from the provided **TalentTreeSO** and populates the **talents** list with new **Talent** objects created from the nodes defined in the **TalentTreeSO**.

Initialize(string treeld)

- This method initializes the talent tree with a specific **treeld**. The **talents** list is initialized but remains empty, allowing for custom talent creation or loading later.

OverrideTalents(List<TalentTreeNodeSO> talentTreeNodes)

- This method clears the existing talents in the tree and replaces them with a new set of talents created from the provided **TalentTreeNodeSO** objects. It's used when you need to completely replace the talents in a tree.

ClearTalents()

- Clears the current list of talents in the tree, essentially resetting the tree's state.

FindTalentByID(int id)

- Searches for and returns a **Talent** object with the specified **id** from the **talents** list. If no matching talent is found, it returns **null**.

TryAddTalentToApplyList(Talent talent)

- Attempts to add a talent to the temporary apply list. This method checks if there are enough talent points available and if the talent hasn't reached its maximum points. If these conditions are met, the method deducts the required points, updates the talent's availability, and adds the talent to the apply list.

AddTalentToTempApplyList(Talent talent)

- Adds a talent to the temporary list of talents that are pending application. This method is used during the editing phase when changes are not yet committed.

ApplyTalent()

- Applies all the talents in the apply list, making their changes permanent. It finalizes the points allocation and updates the talents to reflect these changes.

CancelApply()

- Reverts the changes made during the editing phase by restoring the talent points and availability to their previous state before any modifications. This effectively cancels any uncommitted talent changes.

StopEditing()

- Stops the editing phase by clearing the apply list and invoking the `IsEditing` event to signal that editing has concluded.

AddPoints(int amount)

- Adds a specified amount of talent points to the tree, increasing the available points for allocation.

RemovePoints(int amount)

- Removes a specified amount of talent points from the tree, decreasing the available points for allocation.

CreateTalent(int talentId, TalentTreeNodeSO talentTreeNodeSO)

- Creates a new `Talent` object using the provided `talentId` and `TalentTreeNodeSO` data. The new talent is then added to the `talents` list in the tree.

LoadTree(TalentTreeSaveData talentTreeSaveData)

- Loads the state of the tree from a `TalentTreeSaveData` object. It sets the talent points and updates each talent's points and availability according to the saved data.

TalentTreeUiPresenter Class:

Initialize()

- This method initializes the UI for the active talent tree. It sets up the talent points display and assigns the necessary event listeners to the Apply and Cancel buttons. It ensures the UI reflects the current state of the active talent tree.

AddPoints(int value)

- This method is used to add a specified number of points (**value**) to the active talent tree. It directly modifies the points allocated to the active talent tree, allowing for dynamic adjustments to the talent points.

OnApplyButtonClick()

- This method is triggered when the Apply button is clicked. It commits the changes made to the talents in the active talent tree, applying all pending talent allocations.

OnCancelButtonClick()

- This method is triggered when the Cancel button is clicked. It reverts any uncommitted changes made to the talents in the active talent tree, effectively cancel any pending modifications.

OnTalentChange(bool isEditing)

- This method responds to changes in the editing state of the talent tree. It updates the interactability of the Apply and Cancel buttons based on whether the tree is currently being edited (**isEditing**).

OnTalentPointsChanged(int talentPoints)

- This method updates the displayed number of talent points in the UI whenever the talent points in the active tree change. It ensures that the user interface reflects the correct number of available points.